1 协程

1.1 协程的实现

- 协程不是计算机提供，程序员人为创造的。
- 协程（Coroutine）,也可以被称微线程，是医用用户态内的上下文切换技术。简而言之，其实就是通过一个线程实现代码块相互切换执行。例如

```
def func1():
    print(1)
    ...
    print(2)

def func2():
    print(3)
    ...
    print(4)

func1()
func2()
```

实现协程有以下几种方法

1. `greenlet` （早期模块）
2. `yield` 关键字
3. `asyncio` 装饰器（python 3.4 之后）
4. `async` 、 `await` 关键字（python 3.5 之后） **【官方推荐】**

其中，主要使用的时 `greenlet` 和 `async`、`await` 关键字这两种

1.1.1 greenlet实现

- 安装

```
pip3 install greenlet
```

- 代码

```
from greenlet import greenlet

def func1():
    print(1)          # 第2步：输出1
    gr2.switch()      # 第3步：切换到func2函数
    print(2)          # 第6步：输出2
    gr2.switch()      # 第7步：切换到函数func2

```

```python
def func2():
    print(3)        # 第4步：输出3
    gr1.switch()    # 第5步：切换到func1函数
    print(4)        # 第8步：输出4

gr1 = greenlet(func1)
gr2 = greenlet(func2)
gr1.switch() # 第1步：去执行func1函数

"""输出如下"""
1
3
2
4
```

1.1.2 yield关键字

- 通过yield实现的比较牵强，仅仅满足协程的定义，不重要
- 代码

```python
def func1():
    yield 1
    yield from func2()
    yield 2


def func2():
    yield 3
    yield 4

f1 = func1()
for item in f1:
    print(item)

"""输出如下"""
1
3
4
2
```

1.1.3 asyncio

- 不用安装，但是是在 `python3.4` 版本之后

> DeprecationWarning: "@coroutine" decorator is deprecated since Python 3.8, use "async def" instead

- 代码

```python
import asyncio

@asyncio.coroutine
def func1():
    print(1)
    yield from asyncio.sleep(2) # 遇到IO耗时时操作，自动化切换到tasks中的其他任务
    print(2)

@asyncio.coroutine
def func2():
    print(3)
    yield from asyncio.sleep(2) # 遇到IO耗时时操作，自动化切换到tasks中的其他任务
    print(4)


tasks = [
    asyncio.ensure_future(func1()),
    asyncio.ensure_future(func2())
]

loop = asyncio.get_event_loop()
loop.run_until_complete(asyncio.wait(tasks))

"""输出"""
1
3
2
4
```

注意：遇到IO阻塞自动切换

1.1.4 async_await关键字

- 在 `python3.5` 之后
- 代码

```python
import asyncio


async def func1():
    print(1)
    await  asyncio.sleep(2) # 遇到IO耗时时操作，自动化切换到tasks中的其他任务
    print(2)


async def func2():
    print(3)
```

```
    await  asyncio.sleep(2) # 遇到IO耗时时操作，自动化切换到tasks中的其他任务
    print(4)


tasks = [
    asyncio.ensure_future(func1()),
    asyncio.ensure_future(func2())
]

loop = asyncio.get_event_loop()
loop.run_until_complete(asyncio.wait(tasks))

"""输出"""
1
3
2
4
```

1.2 协程意义

- 在一个线程中如果遇到IO等待时间，线程不会傻傻等，利用空闲的时候再去干点其他事
- 案例:去下载三张图片(网络IO)

1.2.1 下载图片(同步)

- 普通方法（用时 0.7880749702453613）

  普通方法又称同步方式，即顺序完成任务

```
import requests
import os
import functools
import time
from loguru import logger
import asyncio

def calc_time(func):
    if asyncio.iscoroutinefunction(func):
        @functools.wraps(func)
        async def warpper(*args,**kwargs):
            logger.info(f'正在下载:{args}')
            start_time = time.time()
            response = func(*args,**kwargs)
            end_time = time.time()
            logger.info(f'下载完成，共用时 {end_time-start_time} 秒')
            return response
        return warpper
    else:
```

```python
        @functools.wraps(func)
        def warpper(*args,**kwargs):
            logger.info(f'正在下载:{args}')
            start_time = time.time()
            response = func(*args,**kwargs)
            end_time = time.time()
            logger.info(f'下载完成，共用时 {end_time-start_time} 秒')
            return response
        return warpper

@calc_time
def download_image(url):
    response = requests.get(url)
    file_name = os.path.basename(url)
    file_path = os.path.join('./save',file_name)
    with open(file_path,mode='wb') as file_object:
        file_object.write(response.content)

if __name__ == "__main__":
    st = time.time()
    url_list = ["https://qinchihongye-1313059842.cos.ap-guangzhou.myqcloud.com/3.png"
                ,"https://qinchihongye-1313059842.cos.ap-guangzhou.myqcloud.com/4.png"
                ,"https://qinchihongye-1313059842.cos.ap-guangzhou.myqcloud.com/5.png"
                ]
    for item in url_list:
        download_image(item)
    logger.info(f"共用时:{time.time() - st} 秒！ ")


"""输出"""
2024-11-09 13:46:17.474 | INFO     | __main__:warpper:22 - 正在下载:('https://qinchihongye-1313059842.cos.ap-guangzhou.myqcloud.com/3.png',)
2024-11-09 13:46:17.731 | INFO     | __main__:warpper:26 - 下载完成，共用时 0.25673913955688477 秒
2024-11-09 13:46:17.731 | INFO     | __main__:warpper:22 - 正在下载:('https://qinchihongye-1313059842.cos.ap-guangzhou.myqcloud.com/4.png',)
2024-11-09 13:46:18.001 | INFO     | __main__:warpper:26 - 下载完成，共用时 0.26875877380371094 秒
2024-11-09 13:46:18.001 | INFO     | __main__:warpper:22 - 正在下载:('https://qinchihongye-1313059842.cos.ap-guangzhou.myqcloud.com/5.png',)
2024-11-09 13:46:18.261 | INFO     | __main__:warpper:26 - 下载完成，共用时 0.25989484786987305 秒
2024-11-09 13:46:18.262 | INFO     | __main__:<module>:46 - 共用时:0.7880749702453613 秒！
```

1.2.2 下载图片(异步)

- 异步方法（用时0.4766538143157959）

  异步方法即，一个任务开始后，不等待它结束，立即进行下一个任务

```python
import aiohttp
import asyncio
import functools
from loguru import logger
import time
import os

def calc_time(func):
    if asyncio.iscoroutinefunction(func):
        @functools.wraps(func)
        async def warpper(*args,**kwargs):
            logger.info(f'正在下载:{args}')
            start_time = time.time()
            response = await func(*args,**kwargs)
            end_time = time.time()
            logger.info(f'下载完成，共用时 {end_time-start_time} 秒')
            return response
        return warpper
    else:
        @functools.wraps(func)
        def warpper(*args,**kwargs):
            logger.info(f'正在下载:{args}')
            start_time = time.time()
            response = func(*args,**kwargs)
            end_time = time.time()
            logger.info(f'下载完成，共用时 {end_time-start_time} 秒')
            return response
        return warpper

@calc_time
async def fetch(session,url):
    async with session.get(url,verify_ssl=False) as response:
        content = await response.content.read()

        file_name = os.path.basename(url)
        file_path = os.path.join('./save',file_name)
        with open(file_path,mode='wb') as file_object:
            file_object.write(content)

async def main():
    async with aiohttp.ClientSession() as session:
        url_list = ["https://qinchihongye-1313059842.cos.ap-guangzhou.myqcloud.com/3.png"
                    ,"https://qinchihongye-1313059842.cos.ap-guangzhou.myqcloud.com/4.png"
```

```
                    ,"https://qinchihongye-1313059842.cos.ap-guangzhou.myqcloud.com/5.png"
                    ]

        tasks = [asyncio.create_task(fetch(session,url)) for url in url_list]

        await asyncio.wait(tasks)

if __name__ == "__main__":
    st = time.time()
    asyncio.run(main())
    logger.info(f"共用时:{time.time() - st} 秒! ")


"""输出"""
2024-11-09 13:47:32.544 | INFO     | __main__:warpper:12 - 正在下载:(<aiohttp.client.ClientSession object at 0x0000013DF78C5B70>, 'https://qinchihongye-1313059842.cos.ap-guangzhou.myqcloud.com/3.png')
2024-11-09 13:47:32.710 | INFO     | __main__:warpper:12 - 正在下载:(<aiohttp.client.ClientSession object at 0x0000013DF78C5B70>, 'https://qinchihongye-1313059842.cos.ap-guangzhou.myqcloud.com/4.png')
2024-11-09 13:47:32.710 | INFO     | __main__:warpper:12 - 正在下载:(<aiohttp.client.ClientSession object at 0x0000013DF78C5B70>, 'https://qinchihongye-1313059842.cos.ap-guangzhou.myqcloud.com/5.png')
2024-11-09 13:47:33.001 | INFO     | __main__:warpper:16 - 下载完成，共用时 0.289186954498291 秒
2024-11-09 13:47:33.016 | INFO     | __main__:warpper:16 - 下载完成，共用时 0.4716525077819824 秒
2024-11-09 13:47:33.018 | INFO     | __main__:warpper:16 - 下载完成，共用时 0.30765700340270996 秒
2024-11-09 13:47:33.020 | INFO     | __main__:<module>:54 - 共用时:0.4766538143157959 秒!
```

2 asyncio模块

2.1 事件循环

- 理解成一个死循环，去检测并执行某些代码

```
# 伪代码
任务列表[任务1，任务2，任务3。。。 ]

while True:
    可执行任务列表，已完成任务列表 = 检查任务列表所有任务，返回 可执行、已完成任务

    for 就绪任务 in 可执行任务列表:
      执行已就绪的任务

    for 已完成任务 in 已完成的任务列表:
      在任务列表中移除已完成的任务

    如果 任务列表 中的任务都已完成，则终止循环
```

- 代码

```
import asyncio

#去生成或获取一个事件循环
loop = asyncio.get_event_loop()
# 将任务放到'任务列表'
loop.run_until_complete(任务)
```

2.2 快速上手

- 什么是协程函数?
    1. 定义一个协程函数：定义函数时候 `async def` 函数名（python 3.5及之后）。
    2. 协程对象，执行协程函数（）得到的协程对象。

```
async def func(): # 定义协程函数
    pass

result= func() # 得协程对象
```

注意：执行协程函数创建协程对象，函数内部代码不会执行

- 让事件循环去执行协程对象

python3.7之前

```python
import asyncio
async def func(): # 定义协程函数
    print('这是一个协程函数的内部')

result= func() # 得协程对象

#去生成或获取一个事件循环
loop = asyncio.get_event_loop()
# 将任务放到‘任务列表’
loop.run_until_complete(result)
```

python3.7以及之后

```python
import asyncio
async def func(): # 定义协程函数
    print('这是一个协程函数的内部')

result= func() # 得协程对象

# loop = asyncio.get_event_loop()
# loop.run_until_complete(result)
asyncio.run(result)
```

2.3 await

- await就是等，等await后面的执行玩了再继续
- await + 可等待的对象（协程对象，Future、Task对象），可等待对象只能是这三种，目前可以都理解为IO等待
- 示例：

```python
import asyncio

async def others():
    print('start')
    await asyncio.sleep(2)
    print('end')
    return "返回值"

async def func():
    print("执行协程函数内部代码")
    # 遇到IO挂起，等IO操作完成之后再继续往下执行。在当前协程挂起时，事件循环可取执行其他协程（任务）
    response1 = await others()
    print("IO请求结束，结果1为：",response1)

    response2 = await others()
    print("IO请求结束，结果2为：",response2)

```

```
asyncio.run(func())


"""输出"""
执行协程函数内部代码
start
end
IO请求结束，结果为：  返回值
start
end
IO请求结束，结果为：  返回值
```

await就是等待对象的值得到后结果之后再继续往下走(注意上面的代码是顺序执行的)

2.4 Task对象（理解）

- 在 事件循环中添加多个任务的。
- Tasks用于并发调度协程, 通过 `asyncio.create_task（协程对象）` 的方式 `创建Task对象` , 这样可以让协程**立即加入事件循环**中等待被调度执行。除了使用 `asyncio.create_task()` 函数以外, 还可以用低层级的 `loop.create_task()` 或 `ensure_future()` 函数。**不建议手动实例化 Task 对象**。

  注意： `asyncio.create_task()` 函数在python3.7中被加入。在python3.7之前，可以改用底层及的 `asyncio.ensure_future()` 函数。
- 示例1

  创建事件循环后，再将任务添加到事件循环

  asyncio.run会创建事件循环

> # loop = asyncio.get_event_loop()
> # loop.run_until_complete(result)
> asyncio.run(result)

```
import asyncio

async def func():
    print(1)
    await asyncio.sleep(2)
    print(2)
    return "返回值"


# async def main():
#     print("main开始")

#     # 创建Task对象，将当前执行func函数任务立即添加到事件循环
#     task1 = asyncio.create_task(func())
```

```
#     # 创建Task对象，将当前执行func函数任务立即添加到事件循环
#     task2 = asyncio.create_task(func())

#     print("main结束")

#     ret1 = await task1
#     ret2 = await task2
#     print(ret1,ret2)

# asyncio.run(main())


async def main():
    print("main开始")

    task_list = [
        asyncio.create_task(func()),
        asyncio.create_task(func())
    ]

    print("main结束")

    done,pending = await asyncio.wait(task_list,timeout=None) # await后面只能是 协程对象，Future、Task对象
    print(done)

asyncio.run(main()) # asyncio.run已经创建事件循环

```

- 示例2

```
import asyncio

async def func():
    print(1)
    await asyncio.sleep(2)
    print(2)
    return "返回值"

task_list = [
    asyncio.create_task(func()),
    asyncio.create_task(func())
]

done,pending = asyncio.run(asyncio.wait(task_list))
print(done)
```

如果如上面的写，会报错，因为Task对象创建会立即加入到事件循环，但是在 `asyncio.create_task(func())` 之前并没有事件循环，这种应该改为

```
import asyncio

```

```
async def func():
    print(1)
    await asyncio.sleep(2)
    print(2)
    return "返回值"

task_list = [
  func(),
    func()
]

done,pending = asyncio.run(asyncio.wait(task_list))
print(done)
```

2.5 asyncio.Future对象(理解)

- Future本质上不会手动使用，一般使用Taks对象，主要理解就行
- 更低级的接口，Task继承Future,Task对象内部await结果的处理基于Future对象来的
- 示例1:

  一直等待

```
import asyncio

async def main():
    # 获取当前事件循环
    loop = asyncio.get_running_loop()
    # 创建一个任务(Future对象),这个任务什么都不干。
    fut = loop.create_future()
    # 等待任务最终结果(Future对象),没有结果则会一直等下去。
    await fut
asyncio.run( main() )
```

- 示例2:

```
import asyncio
async def set_after(fut):
    await asyncio.sleep(2)
    fut.set_result("666")
async def main():
    # 获取当前事件循环
    loop = asyncio.get_running_loop()
    # 创建一个任务(Future对象),没绑定任何行为,则这个任务永远不知道什么时候结束。
    fut = loop.create_future()
    # 创建一个任务(Task对象),绑定了set_after函数,函数内部在2s之后,会给fut赋值。
    # 即手动设置future任务的最终结果,那么fut就可以结束了。
    await loop.create_task(set_after(fut))
```

```
    # 等待 Future对象获取 最终结果,否则一直等下去
    data = await fut
    print(data)
asyncio.run(main())
```

2.6 concurrent.futures.Future对象（重要）

- 使用线程池、进程池实现异步操作时采用到的对象。与上面的asyncio.Future对象没有半毛钱关系

```
import time
from concurrent.futures import Future
from concurrent.futures.thread import ThreadPoolExecutor
from concurrent.futures.process import ProcessPoolExecutor



def func(value):
    time.sleep(1)
    print(value)

# 创建线程池
pool = ThreadPoolExecutor(max_workers=5)


# 创建进程池
# pool = ProcessPoolExecutor(max_workers=5)

for i in range(10):
    fut = pool.submit(func,i)
    print(fut)
```

- 以后写代码可能存在交叉使用(基于协程的异步和基于线程、进程的异步结合)**（重要）**

  使用场景：在用异步编程的时候，如果遇到某个第三方模块不支持基于协程的异步，这时候用 run_in_executor解决问题。

```
import time
import asyncio
import concurrent.futures
def func1():
    # 某个耗时操作
    time.sleep(2)
    return "SB"
async def main():
    loop = asyncio.get_running_loop()

    # 1. Run in the default loop's executor ( 默认 ThreadPoolExecutor )
```

```python
    # 第一步：内部会先调用 ThreadPoolExecutor 的 submit 方法去线程池中申请一个线程去执行func1函数,并返回一个concurrent. futures. Future对象
    # 第二步：调用asyncio.wrap_future将concurrent.futures.Future对象包装为asycio.Future对象。
    # 因为concurrent.futures.Future对象不支持await语法,所以需要包装为asycio.Future对象 才能使用。
    fut = loop.run_in_executor(None, func1)
    result = await fut
    print('default thread pool', result)

    # 2. Run in a custom thread pool:
    with concurrent.futures.ThreadPoolExecutor() as pool:
        result = await loop.run_in_executor(pool, func1)
        print('custom thread pool', result)

    # 3. Run in a custom process pool:
    with concurrent.futures.ProcessPoolExecutor() as pool:
        result = await loop.run_in_executor(pool, func1)
        print('custom process pool', result)

if __name__ == '__main__':
    asyncio.run(main())
```

- 案例：asyncio+不支持异步的模块

```python
import asyncio
import requests

async def download_image(url):
    # 发送网络请求,下载图片(遇到网络下载图片的IO请求,自动化切换到其他任务)
    print("开始下载:", url)
    loop = asyncio.get_event_loop()
    # requests模块默认不支持异步操作,所以就使用线程池来配合实现了。
    future = loop.run_in_executor(None, requests.get, url) # 默认是线程池
    response = await future
    print('下载完成')
    # 图片保存到本地文件
    file_name = url.rsplit('/')[-1]
    with open(file_name, mode='wb') as file_object:
        file_object.write(response.content)

if __name__ == '__main__':
        url_list = [
    'https://qinchihongye-1313059842.cos.ap-guangzhou.myqcloud.com/image-20241113101036186.png',
    'https://www2.autoimg.cn/newsdfs/g30/M01/3C/E2/120x90_0_autohomecar__ChcCSV2BBICAUntfAADjJFd6800429.jpg',
    'https://qinchihongye-1313059842.cos.ap-guangzhou.myqcloud.com/image-20240602010950670.png'
        ]
```

```
        tasks = [ download_image(url) for url in url_list]
        loop = asyncio.get_event_loop()
        loop.run_until_complete( asyncio.wait(tasks) )

```

2.7 异步迭代器

- **async for 必须卸载协程函数里面**
- 什么是异步迭代器
  实现了**__aiter__()**和**__anext__()**方法的对象。**__anext__**必须返回一个 awaitable 对象。async for 会处理异步迭代的**__anext__**方法所返回的可等待对象，直到其引发一个**StopAsyncIteration**异常。由**PEP492**引入。
- 什么是异步可迭代对象?
  可在 **async for** 语句中被使用的对象。必须通过它的**__aiter__()**方法返回回一个 asynchronous iterator。由 **PEP 492** 引入。

```
import asyncio
class Reader(object):
    """ 自定义异步迭代器(同时也是异步可迭代对象) """
    def __init__(self):
        self.count = 0

    async def readline(self):
        # await asyncio.sleep(1)
        self.count += 1
        if self.count == 100:
            return None
        return self.count

    def __aiter__(self):
        return self

    async def __anext__(self):
        val = await self.readline()
        if val == None:
            raise StopAsyncIteration
        return val

async def func():
    obj = Reader()
    async for item in obj: # async for 必须放在协程函数里面
        print(item)

asyncio.run( func() )
```

2.8 异步上下文管理器

- 此种对象通过定义__aenter__() 和 __aexit__()方法来对**async with** 语句中的环境进行控制。由**PEP492**引入
- __aenter__() 和 __aexit__()方法 需要实现 __aenter__() 和 __aexit__()方法
- **async with 必须写在协程函数里面**

```python
import asyncio

class AsyncContextManager:
    def __init__(self):
        self.conn = ConnectionAbortedError

    async def do_something(self):
        # 异步操作数据库
        return 666

    async def __aenter__(self):
        # 异步连接数据库
        self.conn = await asyncio.sleep(1)
        return self

    async def __aexit__(self, exc_type,exc,tb):
        # 异步关闭数据库连接
        await asyncio.sleep(1)

async def func():
    async with AsyncContextManager() as f:
        result = await f.do_something()
        print(result)

asyncio.run( func() )
```

3 uvloop

- 是asyncio是事件循环的替代方案（第三方写的）。这个事件循环的效率 > 默认asyncio的事件循环。
- 安装

```
pip2 install uvloop

```

使用：

```
import asyncio
import uvloop
asyncio.set_event_loop_policy(uvloop.EventLoopPolicy())

# 编写asyncio的代码，与之前写的代码一致

# 内部的事件循环会自动化会变为uvloop
asyncio.run(...)
```

注意：一个asgi -> uvicorn 内部使用的就是uvloop，fastapi快的原因就是因为其支持uvloop。

4 案例

##

- requests是不支持协程的

```
1
```